MATH50009.01 数值算法与案例分析 II Spring 2026

Lecture 2 牛顿法变体与多项式插值 — 2026.03.10

教授：邵美悦

Scribe: 路人甲

目录

1 牛顿法 **1**

1.1 高维牛顿法 1

1.2 牛顿法变体 2

1.3 拟牛顿法 2

1.4 非线性预条件 5

2 多项式插值 **6**

1 牛顿法

1.1 高维牛顿法

考虑求解高维非线性方程组的根。设目标函数为 $F(x) = 0$，其中 $F : \mathbb{R}^n \to \mathbb{R}^n$，其导数（即 Jacobi 矩阵）映射为 $J : \mathbb{R}^n \to \mathbb{R}^{n\times n}$.

为了寻找根的迭代公式，我们在当前迭代点 x_k 处对 $F(x)$ 进行 Taylor 展开。由于我们希望寻找真实根 x_*，因此有：

$$0 = F(x_*) = F(x_k) + J(x_k)\Delta x_k + O(\|\Delta x_k\|).$$

忽略高阶项，于是

$$J(x_k)\Delta x_k = -F(x_k).$$

求解这个线性方程组，我们便直接得到了经典牛顿法的迭代公式：

$$x_{k+1} = x_k - J(x_k)^{-1}F(x_k).$$

很可惜的是，在高维情况下，每一步都精确计算 Jacobi 矩阵及其逆矩阵的计算代价是极其高昂的。求解线性方程组 $J(x_k)\Delta x_k = -F(x_k)$ 的复杂度为 $O(n^3)$，这在大规模问题中是不可接受的。

单论一元牛顿法，每一步求导数可能也很贵，尤其当 f 本身就是一个复杂函数时，计算 $f'(x)$ 可能比计算 $f(x)$ 更加昂贵。因此，在实际应用中，我们需要寻找一些变体来降低每一步的计算成本。

1.2 牛顿法变体

在上节课中，我们提到牛顿法具有局部二次收敛性，而全局收敛性是要具体问题具体分析的。例如，如果对函数 $\arctan x$ 应用牛顿法，我们需要保证初值不能太大：$x_0 \in (-\alpha, \alpha)$（见 3 月 3 日作业）。如果对于每一个问题，我们都要事先分析初值的范围，那么这个算法无疑是不够稳健的。那么，如何改善算法的稳健性呢？一个自然的扩展是引入步长控制，这就是阻尼牛顿法（Damped Newton's method）：

$$x_{k+1} = x_k - \lambda_k \frac{f(x_k)}{f'(x_k)}, \quad \lambda_k \in (0, 1].$$

为什么引入步长控制能够有效避免迭代发散呢？以 arctan 函数为例。我们知道，当 $|x_0| > \alpha$ 时，每一步迭代产生的切线斜率绝对值 $|f'(x_k)|$ 都会越来越小，与 x 轴交点 x_{k+1} 的绝对值越来越大，最终导致迭代发散。引入步长控制后，每一步迭代我们都不再走到切线与坐标轴的交点位置，而是走到一半就停下来了，如果停下的位置正好落在收敛区间内，那么后续迭代就可以正常进行下去。

1.3 拟牛顿法

既然精确求解导数如此昂贵，那么，我们能否避开导数的精确计算？这便是拟牛顿法（Quasi-Newton method）的核心动机：通过某种方式近似导数。

简化牛顿法 (Simplified Newton's Method) 最简单的策略是简化牛顿法：我们在迭代过程中固定使用初始点的导数：$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_0)}$. 只要初始点 x_0 足够接近精确解 x_*，该方法就可以收敛，但代价是其通常只具有线性收敛速度。严格的分析如下：

我们考察其不动点迭代函数 $g(x) = x - \frac{f(x)}{f'(x_0)}$。求导可得

$$g'(x) = 1 - \frac{f'(x)}{f'(x_0)} = \frac{f'(x_0) - f'(x)}{f'(x_0)}.$$

在精确解处的导数绝对值为：

$$|g'(x_*)| = \left| \frac{f'(x_0) - f'(x_*)}{f'(x_0)} \right|.$$

一个关键的观察是，当且仅当初始导数恰好等于精确解处的导数，即 $f'(x_0) = f'(x_*)$ 时，系统具有局部平方收敛速度。但倘若 $f'(x_0)$ 与 $f'(x_*)$ 之间存在差距，那么迭代的收敛速度将退化为线性，甚至可能发散。

Steffensen's Method 那么，如果不固定导数，我们还能如何近似呢？Steffensen 方法提供了一个思路：利用函数自身的值来进行有限差分近似。我们知道，函数 f 在某点 x 处的导数定义为

$$f'(x) = \lim_{h\to 0} \frac{f(x+h) - f(x)}{h}.$$

当我们使用牛顿法进行迭代时，我们能够相信迭代的后期 $f(x_k)$ 会比较小，因此我们可以将精确导数 $f'(x_k)$ 替换为：

$$f'(x_k) \approx \frac{f(x_k + f(x_k)) - f(x_k)}{f(x_k)}.$$

代入迭代公式后，我们得到：

$$x_{k+1} = x_k - \frac{(f(x_k))^2}{f(x_k + f(x_k)) - f(x_k)}.$$

该方法具有局部的二次收敛性。然而必须指出，有时求函数值也会很贵——在 3 月 3 日作业中，我们知道 Lambert W–函数的求值比一些初等函数要复杂得多。倘若函数的求值比较复杂，以至于占据了主要的计算资源，此处一步迭代算了两次函数值，相当于原来两次迭代的计算量。原本的局部二次收敛，现在将演变为

$$\Delta_{k+2} \le C\Delta_k^2$$

假如我们假设

$$\Delta_{k+1} \approx \tilde{C}\Delta_k^{\alpha}$$

则有

$$\Delta_{k+2} \approx \tilde{C}\Delta_{k+1}^{\alpha} \approx \tilde{C}^{1+\alpha}\Delta_k^{\alpha^2}.$$

因此 $\alpha = \sqrt{2}$，也就是说，当函数求值占据主要的复杂度时，Steffensen 方法的收敛阶为 $\sqrt{2}$，介于线性和二次之间。

割线法 (Secant Method) 进一步地，如果想充分利用已有的迭代路径，我们可以采用割线法，利用前两步的历史数据来计算差商：

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

该方法的收敛阶为黄金分割比 $\frac{1+\sqrt{5}}{2}$。收敛阶的证明需要利用插值技术，下面我们简要说明。读者如有不解之处，可先行跳过，待系统学习插值后再回头查看。

证明. 割线法本质上是用在 $(x_k, f(x_k))$ 和 $(x_{k+1}, f(x_{k+1}))$ 两点的插值函数 $p(x)$ 来近似原函数，有

$$f(x) = p(x) + f[x_k, x_{k+1}, x](x - x_k)(x - x_{k+1}). \tag{1}$$

由于我们是在两个点上插值，得到的插值函数 $p(x)$ 必然是一次函数。现在，我们的目标便是找出 $|x_{k+2} - x_*|, |x_{k+1} - x_*|, |x_k - x_*|$ 之间的递推关系。

我们将 x_* 代入 (1) 中，有

$$0 = f(x_*) = p(x_*) + f[x_k, x_{k+1}, x_*](x_* - x_k)(x_* - x_{k+1}).$$

由割线法定义，我们知道 $p(x_{k+2}) = 0$. 结合 p 是一次函数，我们知道

$$p(x_{k+2}) - p(x_*) = k(x_{k+2} - x_*) = -p(x_*).$$

又由

$$-p(x_*) = f[x_k, x_{k+1}, x_*](x_* - x_k)(x_* - x_{k+1})$$

有

$$k(x_{k+2} - x_*) = f[x_k, x_{k+1}, x_*](x_* - x_k)(x_* - x_{k+1}),$$

即

$$C|x_{k+2} - x_*| = (C|x_{k+1} - x_*|)(C|x_k - x_*|).$$

令 $\Delta_k = C|x_k - x_*|$，两边取对数，有

$$\ln \Delta_{k+2} = \ln \Delta_{k+1} + \ln \Delta_k.$$

显然

$$\ln \Delta_k = \alpha\phi_1^k + \beta\phi_2^k \approx \alpha_1\phi_1^k,$$

其中 $\phi_1 = (1+\sqrt{5})/2$, $\phi_2 = (1-\sqrt{5})/2$. 于是

$$\ln \Delta_{k+1} \approx \phi_1 \ln \Delta_k,$$

亦即

$$\Delta_{k+1} \approx \Delta_k^{\phi_1}$$

这说明割线法的收敛阶是黄金分割率 $\frac{1+\sqrt{5}}{2}$. □

Broyden 方法 当我们把割线法的思想推广到高维时，便引出了著名的 Broyden 方法。Broyden 方法背后的思想是，最多在第一次迭代时计算整个 Jacobi 矩阵，在其余迭代中做秩一修正。

在割线法中，我们需要求解割线方程 $f_k' \cdot (x_k - x_{k-1}) = f(x_k) - f(x_{k-1})$. 类似地，在高维问题中，我们的矩阵需要满足割线条件：

$$J_k(x_k - x_{k-1}) = f(x_k) - f(x_{k-1}).$$

为清晰起见，下面我们记 $\Delta x_k = x_k - x_{k-1}$，记 $\Delta f_k = f(x_k) - f(x_{k-1})$，则割线条件可以简化为：

$$J_k \Delta x_k = \Delta f_k.$$

现在的问题是：我们已知 Δf_k 和 Δx_k，以及上一步的 J_{k-1}，要怎么求解 J_k 呢？显然直接求是不可行的，我们需要对 J_{k-1} 做更新，以逼近 J_k. 假设 $J_k = J_{k-1} + \Delta J_k$，我们现在就要求出 ΔJ_k，使得更新后的 J_k 满足割线条件。

考虑线性系统 $Ax = b$. 假如计算解 $\hat{x}$ 并不等于真实解，我们可以考虑“泼脏水”——引入一个向后误差到系数矩阵项，即 $(A + \Delta A)\hat{x} = b$，现在我们就要求出这个 ΔA. 但是，ΔA 的选择太多了，我们必须引入一些约束条件来缩小选择范围。一个自然的约束是秩约束：$\operatorname{rank}(\Delta A) \leq 1$. 于是，我们可以把 ΔA 表示为 $\Delta A = uv^*$. 于是，残差可以表示为

$$r = b - A\hat{x} = \Delta A\hat{x} = uv^*\hat{x}.$$

显然 r 与 u 线性相关，因此我们可以选择 $u = r$. u 的选择相对确定，而 v 的选择就有很多了，v 只需满足 $v^*x = 1$ 即可。我们需要再度引入约束条件。一个合理的条件是使得扰动矩阵的 Frobenius 范数最小化。由于 $\|\Delta A\|_{\mathrm{F}} = \|u\|_2 \cdot \|v\|_2$，于是这个问题就转化为

$$\min \|v\|_2 \quad \text{s.t.} \quad v^*\hat{x} = 1.$$

于是我们选取 $v = \frac{\hat{x}}{\|\hat{x}\|_2^2}$. 于是，

$$\Delta A = \frac{(b - A\hat{x})\hat{x}^*}{\|\hat{x}\|_2^2}.$$

类似地，在非线性迭代中，Jacobian 矩阵的递推公式可以构造为：

$$J_k = J_{k-1} + \frac{(\Delta f_k - J_{k-1}\Delta x_k)\Delta x_k^\top}{\|\Delta x_k\|_2^2}.$$

从代数直观上，这一步意味着我们在所有满足多维割线方程的矩阵中，寻找与上一步 Jacobian 矩阵距离最近的一个。

但是，牛顿法需要的是 J_k^{-1}，由于我们做的是低秩修正，我们可以利用 Sherman–Morrison–Woodbury 公式直接更新逆矩阵，从而规避昂贵的矩阵求逆运算。这种方法叫 “Good Broyden’s method”.

注：Sherman–Morrison–Woodbury 公式：

$$(A - XB^{-1}Y^*)^{-1} = A^{-1} + A^{-1}X(B - Y^*A^{-1}X)^{-1}Y^*A^{-1}.$$

利用该公式，我们可以直接更新 J_k^{-1}，而不需要先求 J_k 再求逆。

此外，我们也可以走另一条更直接的路径：直接对逆矩阵 J_k^{-1} 进行秩一更新，这就是所谓的 “Bad Broyden’s method”. 当然，在工程实践中，“bad”不一定坏，“good”不一定好。

1.4 非线性预条件

我们知道，预条件可以加速收敛。其核心思路是降维打击：固定一部分未知数，去更新其余未知数。

2 多项式插值

在许多实际问题中，函数 $f(x)$ 可能没有简单的解析表达式，或者计算复杂，但我们能通过实验或观测得到它在某些点上的取值。给定平面上 n 个互不相同的点 $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n) \in \mathbb{R}^2$，其中 x_i 互不相同。我们希望构造一个简单易算的函数 $p(x)$ 来近似替代 f，并要求

$$p(x_i) = y_i, \quad i = 1, 2, \ldots, n.$$

这类问题称为插值问题，$p(x)$ 称为插值函数。本节我们主要关注多项式插值。我们的目标是寻找一个多项式 $p \in \mathbb{R}[x]$ 满足以下条件：

- $p(x_i) = y_i$
- $\deg p(x) \leq n - 1$.

一般的多项式表达结构可以写为：

$$\tilde{p}(x) = p(x) + \left(\prod_{i=1}^{n} (x - x_i) \right) q(x).$$

也就是说，所有满足第一个条件的多项式，都可以表示为一个特解 $p(x)$ 加上通解 $\left(\prod_{i=1}^{n}(x - x_i)\right) q(x)$，其中 $q(x)$ 是任意多项式。由于我们要求 $\deg p(x) \leq n - 1$，因此 $q(x)$ 必须是零多项式。由此，我们得到了插值多项式的唯一性。

从另一个角度看，如果采用待定系数法： $a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1} = y$ 则我们需要求解如下的 Vandermonde 线性系统：

$$\begin{bmatrix} 1 & x_1 & \ldots & x_1^{n-1} \\ 1 & x_2 & \ldots & x_2^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \ldots & x_n^{n-1} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

虽然该矩阵的行列式 $\det = \prod_{j>i}(x_j - x_i) \neq 0$，从而保证了解的唯一性。但这种直接解法面临的问题是，Vandermonde 系统是高度病态的系统，此法得到的解往往不够精确。我们需要另寻他法。

Lagrange 方法 记 $\mathbb{P}_{n-1}$ 为次数不超过 n 的多项式的全体。对每个节点 x_i，我们定义取值泛函 $L_i : \mathbb{P}_n \to \mathbb{R}$ 为

$$L_i(p) = p(x_i), \forall i = 1, 2, \ldots, n.$$

显然每个 L_i 都是线性泛函。进一步地，这族线性泛函线性无关。因此它们构成了对偶空间 $\mathbb{P}_{n-1}^*$ 的一组基。为了显式地构造出插值多项式，我们可以在 $\mathbb{P}_{n-1}$ 中找一组基函数 $\{l_i(x)\}_{i=1}^{n}$，使得它

们对取值泛函构成对偶基，即满足

$$l_i(x_j) = \delta_{i,j} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

一旦找到这样的基函数，插值多项式便可直接写为它们的线性组合

$$p(x) = \sum_{i=1}^{n} y_i \cdot l_i(x).$$

那么这组基该如何构造呢？由于 $l_i(x)$ 在 $x_j (j \neq i)$ 处为零，而 $l_i(x)$ 是次数不超过 $n-1$ 的多项式，因此它必定具有如下形式

$$l_i(x) = C_i \prod_{j \neq i} (x - x_j).$$

再利用 $l_i(x_i) = 1$，我们可以确定常数 C_i 的值：

$$C_i = \frac{1}{\prod_{i \neq j}(x_i - x_j)} \implies l_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

通过这组基底的线性组合，便得到了整体插值多项式：

$$p(x) = \sum_{i=1}^{n} y_i \cdot l_i(x). \quad O(n^2)$$

这个公式称为 Lagrange 插值公式。它结构清晰，便于理论分析，且直接体现了线性空间中对偶基的思想。

2.1 例

已知函数 $f(x)$ 满足 $f(1) = 1$，$f(2) = 4$，$f(3) = 9$，求 $f(0)$ 的近似值。

解. 我们先计算 *Lagrange* 基底函数：

$$l_1(x) = \frac{(x-2)(x-3)}{(1-2)(1-3)}$$
$$l_2(x) = \frac{(x-1)(x-3)}{(2-1)(2-3)}$$
$$l_3(x) = \frac{(x-1)(x-2)}{(3-1)(3-2)}$$

代入插值公式：

$$p(0) = 1 \cdot l_1(0) + 4 \cdot l_2(0) + 9 \cdot l_3(0) = 0.$$

现在，我们来分析一下 Lagrange 插值的计算复杂度。对给定的 x，我们计算 $l_i(x)$ 的复杂度是 $\Theta(n)$，因此计算 $p(x)$ 的复杂度是 $\Theta(n^2)$.

可以注意到，上述计算依然可以优化。我们发现基底的分母 $x_i - x_j$ 有很多重复计算，分子 $x - x_j$ 也有很多重复。为了改进，我们将其变形为重心 Lagrange 插值公式：

$$p(x) = \omega_n(x) \sum_{i=1}^{n} \frac{\lambda_i y_i}{x - x_i}$$

其中

$$\omega_n(x) = \prod_{i=1}^{n} (x - x_i)$$
$$\lambda_i = \left(\prod_{j \neq i} (x_j - x_i) \right)^{-1}$$

这个改写非常漂亮，它将依赖于 x 的动态部分与仅依赖于节点的静态部分剥离开来。在计算复杂度上它也极具优势：我们需要 $\Theta(n)$ 次预备运算，然后每次计算只需要 $\Theta(n)$ 次在线求值。

Neville 方法 Lagrange 插值法虽形式优美，理论清晰，但有一个小缺点：若需要增加一个新节点，所有的基函数 $l_i(x)$ 都需要重新计算，之前的计算成果无法保留。接下来我们探讨 Neville 方法。它提供了一个几何意义极强的递归视角：它并不给出多项式的显式表达式，而是通过两个低次插值多项式的仿射组合，逐步生成高次插值多项式在某一点的值，说得通俗些就是“两条直线生成一条抛物线”。这在只需要计算单个点的插值结果时非常高效。

我们记 $p_{i,j}(x)$ 为过节点 $x_i, x_{i+1}, \ldots, x_j$ 的次数不超过 $j - i$ 的多项式。特别地，$p_{i,i}(x) = y_i$. 我们的目标是求出 $p_{1,n}(x_*)$，即过所有节点的插值多项式在 x_* 处的值。下面，我们希望通过递归的方式，构造出目标插值不等式。具体地，我们希望由 $p_{1,n-1}$ 和 $p_{2,n}$ 构造出 $p_{1,n}$. 现在需要解决的一个问题是：$p_{1,n-1}$ 和 $p_{2,n}$ 的次数最多为 $n-2$，而 $p_{1,n}$ 的次数则有可能达到 $n-1$，因此我们需要在 $p_{1,n-1}$ 和 $p_{2,n}$ 的基础上叠加一个修正项来提升次数。最简单的修正就是仿射组合：

$$p_{1,n}(x) = \alpha(x) p_{1,n-1}(x) + \beta(x) p_{2,n}(x), \quad \alpha(x) + \beta(x) = 1.$$

于是

$$y_1 = \alpha(x_1) y_1 + \beta(x_1) p_{2,n}(x_1) \implies \alpha(x_1) = 1, \beta(x_1) = 0$$
$$y_n = \alpha(x_n) p_{1,n-1}(x_n) + \beta(x_n) y_n \implies \alpha(x_n) = 0, \beta(x_n) = 1$$

由于 $\alpha(x)$ 和 $\beta(x)$ 都是一次多项式，因此我们可以直接写出它们的表达式：

$$\alpha(x) = \frac{x - x_n}{x_1 - x_n}, \quad \beta(x) = \frac{x - x_1}{x_n - x_1}.$$

由此便形成了著名的 Neville 行列式递推公式：

$$P_{1,2,\ldots,n}(x) = \det \begin{bmatrix} x - x_1 & p_{1,n-1}(x) \\ x - x_n & p_{2,n}(x) \end{bmatrix} / (x_n - x_1)$$

该递推算法的整体复杂度为 $\Theta(n^2)$。值得一提的是，从数值分析的角度来看，只要待求值点落在插值区间内（$x_1 \le x_2 \le \cdots \le x_n$ 且 $x_1 \le x_* \le x_n$），这种递推就严格对应于凸组合，能极大抑制舍入误差的放大。

牛顿插值法 最后，我们介绍牛顿插值法。它基于一种增量式构造的思想：每增加一个节点，我们在原多项式上叠加一个修正项。假设对于前 $n-1$ 个节点，已有：$p_{n-1}(x_i) = y_i, \quad i = 1, \ldots, n-1$。现在要强行使该多项式过第 n 个点，即满足 $p_n(x_i) = y_i$，我们构造：

$$p_n(x) = p_{n-1}(x) + C \prod_{i=1}^{n-1} (x - x_i)$$

为求出待定常数 C，将 x_n 代入上式得：

$$C = \frac{y_n - p_{n-1}(x_n)}{\prod_{i=1}^{n-1}(x_n - x_i)}$$

展开后不难发现，这正是所谓的差商结构：

$$C = \sum_{i=1}^{n} \prod_{j \ne i} \frac{y_i}{x_i - x_j}$$

牛顿法的代数优势在于它的“继承性”：当需要增加新的样本点时，我们无需像 Lagrange 方法那样推倒重来，只需计算新增的一项差商并将其叠加即可。